



Image, sécurité et deep learning

Compte-rendu TP4

Classification d'images par CNN

Louis Jean

Master 2 IMAGINE
Université de Montpellier
N° étudiant : 21914083

30 septembre 2024

Table des matières

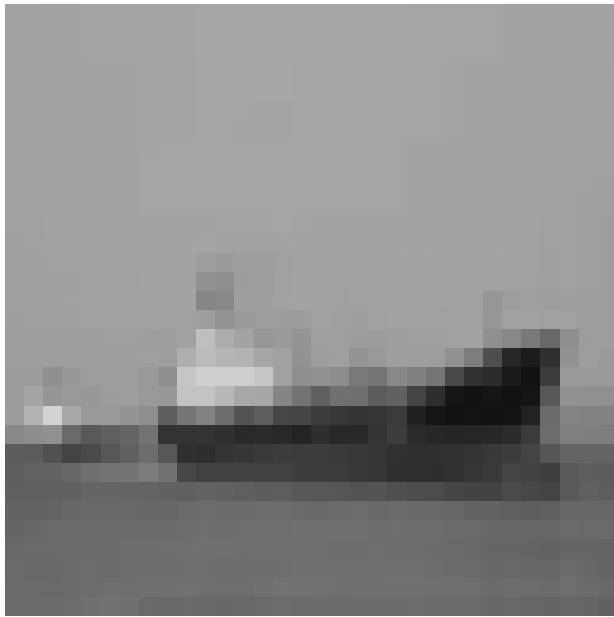
1	Introduction	2
2	Choix de deux classes d'images	2
3	Construction d'un réseau de neurones à 2 couches	2
3.1	Première couche convolutive	2
3.2	Deuxième couche convolutive	3
3.3	Couche fully connected et fonction SoftMax	3
4	Calcul d'un vecteur moyen pour chacune des deux classes	4
4.1	Évaluation du modèle	4
4.2	Interprétation des résultats	5
5	Calcul d'une fonction de perte sur la base d'un PSNR	5
6	Conclusion	5

1 Introduction

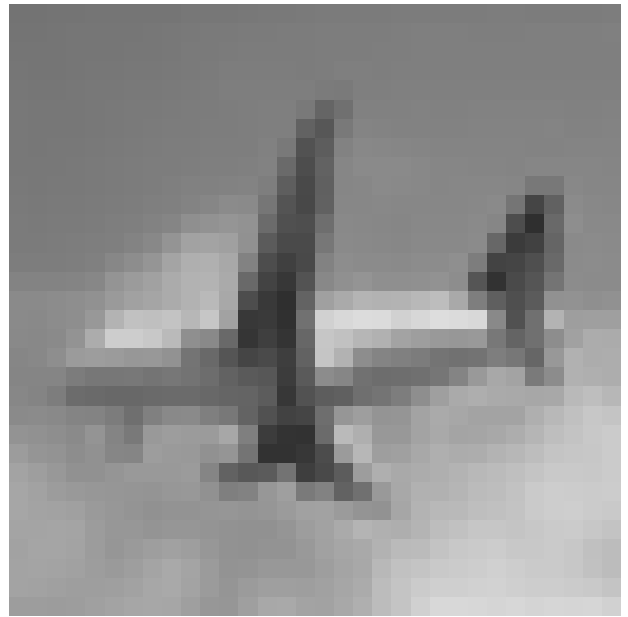
L'objectif de ce TP est de reproduire un réseau de neurones convolutif (CNN) pour la classification d'images appartenant à deux classes distinctes. Le CNN construit comprend deux couches convolutives, suivies d'une couche fully connected, et vise à classer les images en fonction de leur appartenance à l'une des deux classes sélectionnées.

2 Choix de deux classes d'images

J'ai choisi d'utiliser les classes *ship* et *airplane* de la base de données CIFAR-10, contenant des images de taille 32x32. À partir d'un programme C++ pour extraire les images depuis les batches, puis de quelques scripts bash et Python pour renommer, traiter et sélectionner les images, on se retrouve avec chaque classe représentée par 100 images, converties en niveaux de gris et au format PGM. La classe *ship* est la classe 0, tandis que la classe *airplane* est la classe 1.



(a) Exemple d'image de la classe *ship*



(b) Exemple d'image de la classe *airplane*

Figure 1: Exemples d'images des deux classes

3 Construction d'un réseau de neurones à 2 couches

Le CNN est construit avec deux couches convolutives, chacune suivie d'une fonction d'activation ReLU et d'un pooling par max de taille 2×2 . Une couche fully connected est ensuite utilisée pour la classification finale.

3.1 Première couche convolutive

La première couche convolutive utilise cinq filtres de taille 3×3 :

- 2 filtres passe-bas :

$$\text{Filtre passe-bas 1} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

$$\text{Filtre passe-bas 2} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

- 2 filtres passe-haut :

$$\text{Filtre passe-haut 1} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

$$\text{Filtre passe-haut 2} = \begin{bmatrix} -1 & -2 & -1 \\ -1 & 11 & -1 \\ -1 & -2 & -1 \end{bmatrix}$$

- 1 filtre avec des coefficients aléatoires, générés dans le code.

Après la convolution, une fonction d'activation ReLU est appliquée :

$$\text{ReLU}(x) = \max(0, x)$$

Ensuite, un pooling par max de taille 2×2 est effectué, réduisant la taille des cartes de caractéristiques. La sortie de cette couche est un ensemble de cinq cartes de caractéristiques de taille 15×15 .

3.2 Deuxième couche convolutive

La deuxième couche convolutive applique trois filtres de taille 3×3 aux cartes de caractéristiques issues de la première couche :

- 1 filtre passe-bas :

$$\text{Filtre passe-bas 3} = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

- 1 filtre passe-haut :

$$\text{Filtre passe-haut 3} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

- 1 filtre avec des coefficients aléatoires.

Chaque filtre est appliqué à chaque carte de caractéristiques de la couche précédente, ce qui donne un total de $5 \times 3 = 15$ cartes de caractéristiques. Après la convolution, la fonction ReLU est appliquée, suivie d'un pooling par max de taille 2×2 . La sortie est un ensemble de 15 cartes de caractéristiques de taille 6×6 .

3.3 Couche fully connected et fonction SoftMax

Les cartes de caractéristiques sont aplaties en un vecteur de 540 coefficients (15 cartes de 6×6 pixels). Une couche fully connected prend ce vecteur en entrée et produit deux scores correspondant aux deux classes. La fonction SoftMax est utilisée pour convertir ces scores en probabilités :

$$P(y = k \mid x) = \frac{e^{s_k}}{\sum_{i=1}^2 e^{s_i}}$$

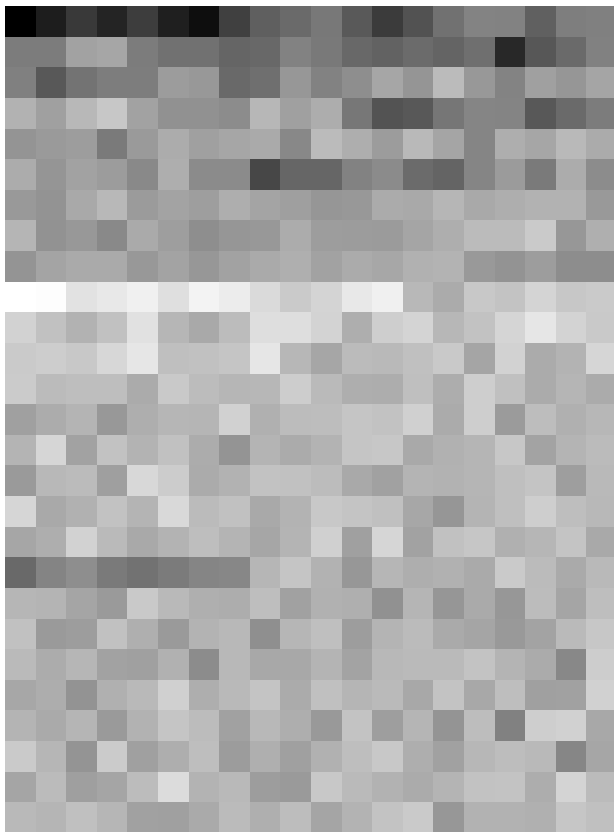
où s_k est le score pour la classe k .

4 Calcul d'un vecteur moyen pour chacune des deux classes

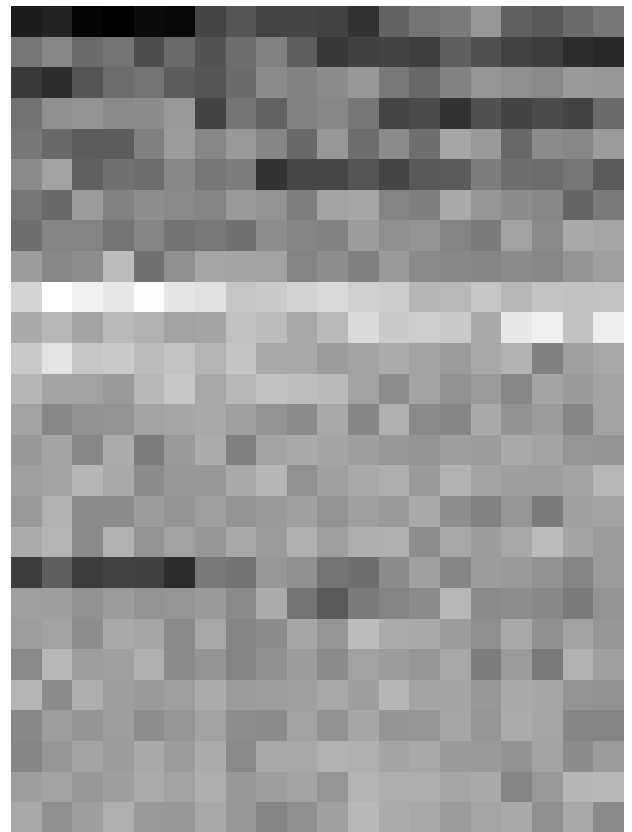
Pour chacune des deux classes, nous avons utilisé 80 images pour calculer le vecteur moyen des caractéristiques. Les étapes sont les suivantes :

1. Passer chaque image à travers le CNN pour obtenir le vecteur de caractéristiques de taille 540
2. Accumuler ces vecteurs pour chaque classe
3. Diviser la somme des vecteurs par le nombre d'images (80) pour obtenir le vecteur moyen de chaque classe

Ces vecteurs moyens représentent les caractéristiques typiques de chaque classe selon le CNN.



(a) Image moyenne de la classe *ship*



(b) Image moyenne de la classe *airplane*

Figure 2: Images moyennes des deux classes

4.1 Évaluation du modèle

Pour évaluer les performances du CNN, nous avons utilisé les 20 images restantes de chaque classe comme ensemble de test. En passant ces images à travers le réseau, nous avons obtenu les prédictions du modèle et avons calculé les métriques suivantes :

- Vrais positifs (VP) : nombre d'images de la classe 1 correctement classées comme classe 1
- Faux positifs (FP) : nombre d'images de la classe 0 incorrectement classées comme classe 1
- Vrais négatifs (VN) : nombre d'images de la classe 0 correctement classées comme classe 0
- Faux négatifs (FN) : nombre d'images de la classe 1 incorrectement classées comme classe 0

Les métriques de performance sont calculées comme suit :

$$\text{Précision} = \frac{VP}{VP + FP} = 0,5$$

$$\text{Rappel} = \frac{VP}{VP + FN} = 1$$

$$F1\text{-score} = 2 \times \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} = 0,667$$

4.2 Interprétation des résultats

Ces résultats montrent que le modèle n'est pas du tout performant, et prédit uniquement la classe *avion*.

5 Calcul d'une fonction de perte sur la base d'un PSNR

Pour évaluer la similarité entre les vecteurs caractéristiques des images de test et les vecteurs moyens, nous avons :

1. Transformé les vecteurs caractéristiques des images de test (20 images restantes par classe) en imagerie de taille 27×20 pixels
2. Normalisé les valeurs des vecteurs pour qu'elles soient entre 0 et 255
3. Calculé le PSNR entre chaque image de test et l'image moyenne de sa classe

Les PSNR moyens obtenus sont :

- PSNR moyen pour la classe 0 (*ship*) : **15.4658** dB
- PSNR moyen pour la classe 1 (*airplane*) : **15.2676** dB

6 Conclusion

Ce TP a permis de construire un CNN simple pour la classification d'images entre deux classes. Les résultats obtenus ne sont pas convaincants mais le travail a permis de comprendre le fonctionnement d'un CNN.

Des améliorations peuvent être apportées pour augmenter les performances du modèle, comme utiliser davantage de données d'entraînement, augmenter le nombre de filtres ou la profondeur du réseau, ou encore implémenter un apprentissage des poids avec une fonction de perte et la rétropropagation.

Merci pour le temps et l'attention que vous avez consacrés à la lecture de ce compte-rendu.